



資料分析與機器學習 (Data Analysis and Machine Learning)

Lecture 2 –

Perceptron learning algorithm & Python basics

教師：陳慶永

Outline

- Artificial Neurons (人工神経元)
- Perceptron (感知器) Learning Algorithm
- Python Basics
- Training a Perceptron Model

Building an understanding of
machine learning algorithms

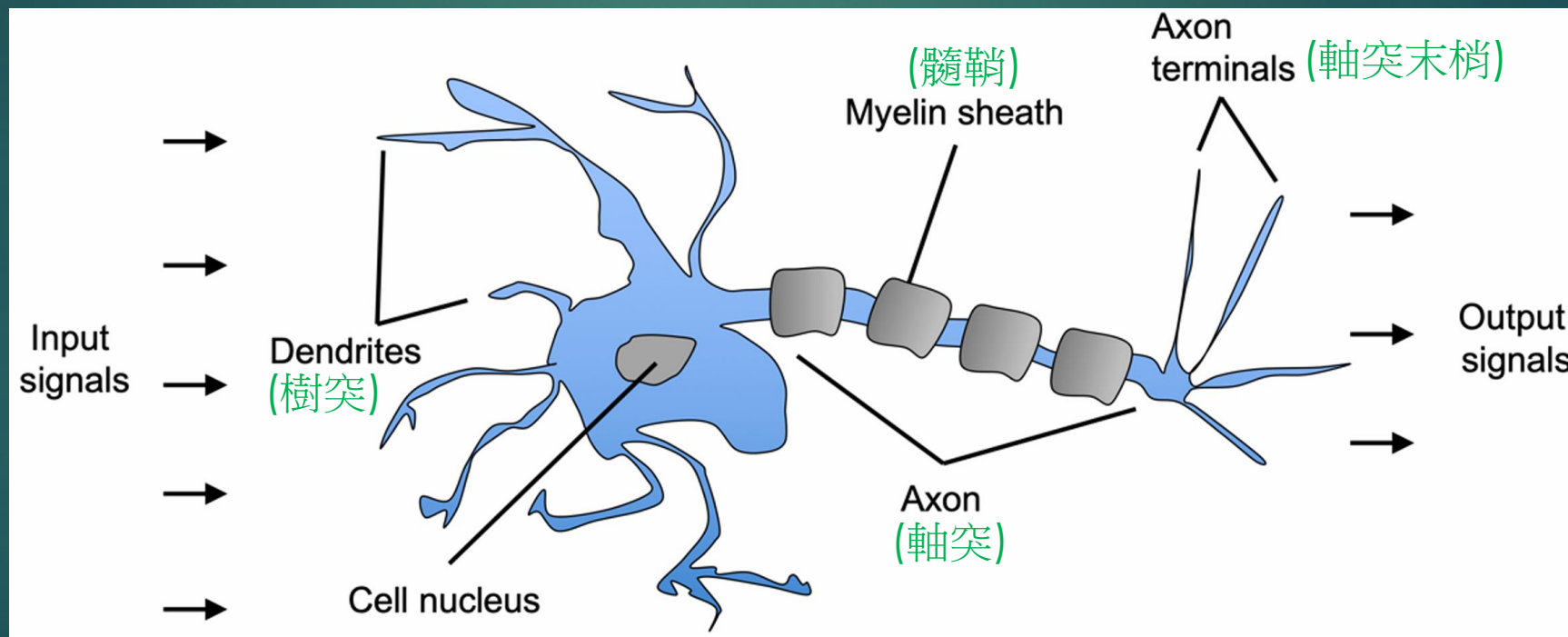
Implementing linear classifiers
for 2-class problems



Artificial Neurons

Neurons in the Brain

- Biological neurons are **interconnected** nerve cells
 - ▶ **Process** and **transmit** chemical and electrical **signals**



Artificial Neurons

■ McCulloch-Pitts (MCP) neuron (1943)

- ▶ Model nerve cell as a simple **logic gate** with **binary outputs**
 - **Multiple signals** arrive at the dendrites, they are then **integrated** into the cell body
 - *If* the accumulated signal **exceeds a certain threshold**, an **output signal is generated** that will be passed on by the axon

■ **Perceptron** (1953)

- ▶ F. Rosenblatt proposed the **perceptron learning rule** based on the MCP neuron

Perceptron

■ Formal definition in Math

► Decision function $\sigma(z)$

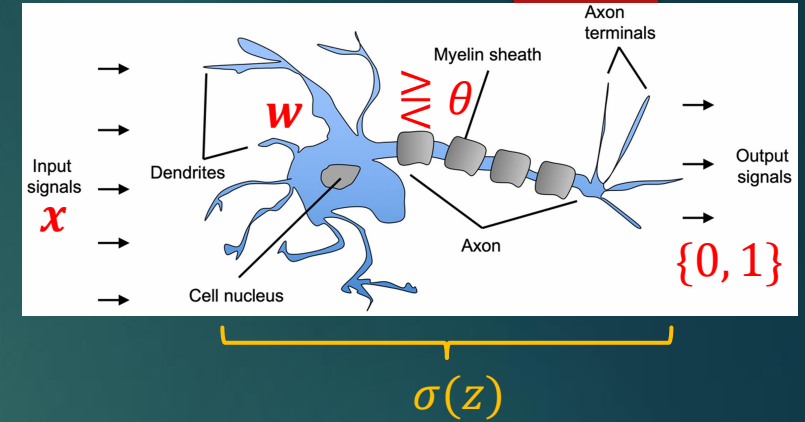
$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

► z is the net input

- Linear combination of inputs $(x_1, x_2, \dots, x_m)$ using weights $(w_1, w_2, \dots, w_m)$

$$z = w_1x_1 + w_2x_2 + \dots + w_mx_m$$

► θ is a threshold ($z - \theta \geq 0$)



Perceptron

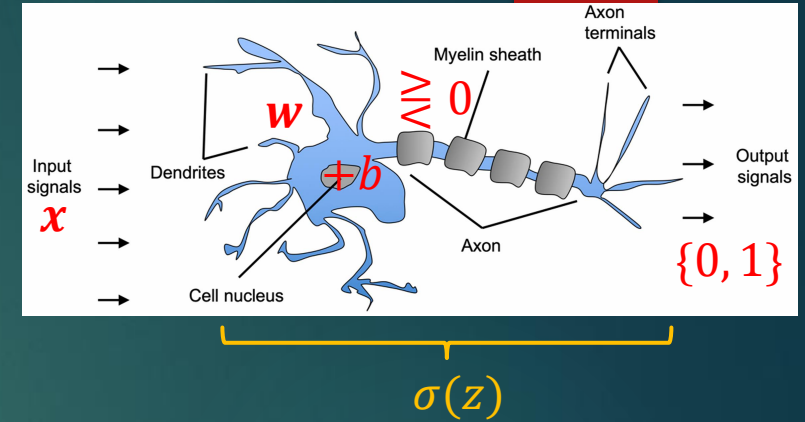
■ Modification of z with bias unit

- ▶ Bias unit b ($= -\theta$) included

$$z = w_1x_1 + \dots + w_mx_m + b = w^T x + b$$

- ▶ Decision function $\sigma(z)$

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

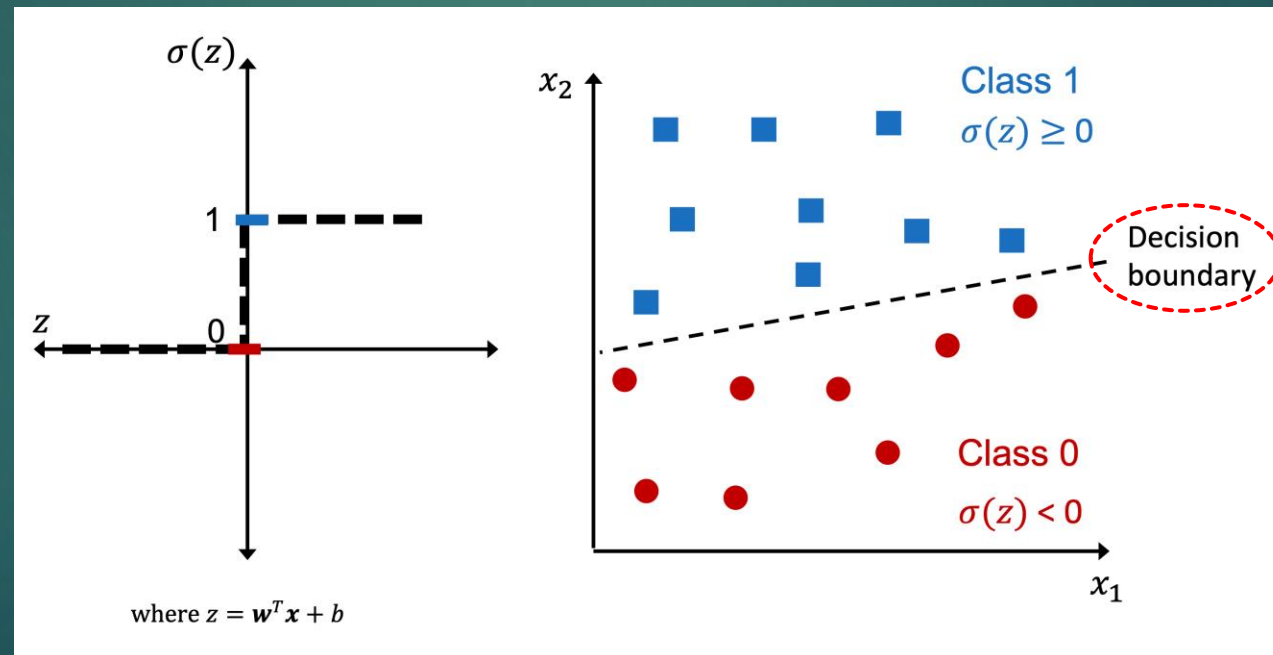


$$w^T = [w_1, w_2, \dots, w_m] \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

Bias unit is essentially an intercept term or negative threshold.

Classification using Perceptron

- The perceptron can be used to discriminate between **two classes** separable by a **linear** decision boundary



Related to $\mathbf{w}$ and b that should be learned from data

Linear Algebra Basics

- **Matrix** (矩陣) is a **2D** array (陣列) of **elements**
 - ▶ Denoted by capital boldface letters (e.g., **A**, **B**, **C**, ...)
 - ▶ An $m \times n$ matrix **A** has **m rows** and **n columns**
 - a_{jk} denotes the element at Row j and Column k

$$\mathbf{A} = [a_{jk}] = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdot & \cdot & \cdots & \cdot \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

n Columns

m Rows

Linear Algebra Basics

■ Vector (向量)

- ▶ A 1D array (i.e., a matrix with only one row or column)
- ▶ Denoted by lowercase boldface letters (e.g., **a**, **b**, **c**, ...)
- ▶ **a** = $[a_j]$ where a_j 's are components of the vector

- Row vector $\mathbf{a} = [a_1 \ a_2 \ \cdots \ a_n]$

- Column vector

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Linear Algebra Basics

■ Examples of matrices and vectors

Square matrix (方陣)

$$\begin{array}{ll} \textcircled{1} \begin{bmatrix} 0.3 & 1 & -5 \\ 0 & -0.2 & 16 \end{bmatrix}, & \textcircled{2} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}, \\ \textcircled{3} \begin{bmatrix} e^{-x} & 2x^2 \\ e^{6x} & 4x \end{bmatrix}, & \textcircled{4} [a_1 \ a_2 \ a_3], \quad \textcircled{5} \begin{bmatrix} 4 \\ \frac{1}{2} \end{bmatrix} \end{array}$$

Column vector
(行向量)

Row vector
(列向量)

Linear Algebra Basics

■ Matrix Transpose (矩陣轉置) 列變行、行變列

Transposition of Matrices and Vectors

If $\mathbf{A} = \begin{bmatrix} 5 & -8 & 1 \\ 4 & 0 & 0 \end{bmatrix}$, then $\mathbf{A}^T = \begin{bmatrix} 5 & 4 \\ -8 & 0 \\ 1 & 0 \end{bmatrix}$.

A little more compactly, we can write

$$\begin{bmatrix} 5 & -8 & 1 \\ 4 & 0 & 0 \end{bmatrix}^T = \begin{bmatrix} 5 & 4 \\ -8 & 0 \\ 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 3 & 0 & 7 \\ 8 & -1 & 5 \\ 1 & -9 & 4 \end{bmatrix}^T = \begin{bmatrix} 3 & 8 & 1 \\ 0 & -1 & -9 \\ 7 & 5 & 4 \end{bmatrix},$$

Furthermore, the transpose $[6 \ 2 \ 3]^T$ of the row vector $[6 \ 2 \ 3]$ is the column vector

$$[6 \ 2 \ 3]^T = \begin{bmatrix} 6 \\ 2 \\ 3 \end{bmatrix}. \quad \text{Conversely,} \quad \begin{bmatrix} 6 \\ 2 \\ 3 \end{bmatrix}^T = [6 \ 2 \ 3].$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

$$\mathbf{w}^T = [w_1, w_2, \dots, w_m]$$

Linear Algebra Basics

■ Matrix Multiplication (矩陣乘法)

$$\begin{matrix} \mathbf{A} & \mathbf{B} & = & \mathbf{C} \\ [m \times n] & [n \times p] & = & [m \times p]. \end{matrix}$$

Matrix Multiplication

Multiplication of rows into columns

$$\mathbf{AB} = \begin{bmatrix} 3 & 5 & -1 \\ 4 & 0 & 2 \\ -6 & -3 & 2 \end{bmatrix} \begin{bmatrix} 2 & -2 & 3 & 1 \\ 5 & 0 & 7 & 8 \\ 9 & -4 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 22 & -2 & 43 & 42 \\ 26 & -16 & 14 & 6 \\ -9 & 4 & -37 & -28 \end{bmatrix}$$

Here $c_{11} = 3 \cdot 2 + 5 \cdot 5 + (-1) \cdot 9 = 22$, and so on. The entry in the box is $c_{23} = 4 \cdot 3 + 0 \cdot 7 + 2 \cdot 1 = 14$.
The product $\mathbf{BA}$ is not defined. ■

Products of Row and Column Vectors

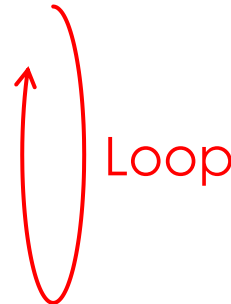
$$\begin{matrix} w^T x = ? \\ [3 & 6 & 1] \end{matrix} \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} = [19], \quad \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} [3 & 6 & 1] = \begin{bmatrix} 3 & 6 & 1 \\ 6 & 12 & 2 \\ 12 & 24 & 4 \end{bmatrix}.$$



Perceptron Learning Algorithm

Perceptron Learning Rule

■ Simple rule of learning $\mathbf{w}$ and b from data

1. Initialize the weights and bias unit to 0 or small random numbers
 2. For each training example, $\mathbf{x}^{(i)}$:
 - a. Compute the output value, $\hat{y}^{(i)}$
 - b. Update the weights and bias unit
- 

- ▶ The output value $\hat{y}^{(i)} = \sigma(z^{(i)})$ (i.e., predicted class label for the i th training sample $\mathbf{x}^{(i)}$)
- ▶ How to update $\mathbf{w}$ and b ? (given the true class label)

Perceptron Learning Algorithm (PLA)

■ An intuitive algorithm: **Mistake-driven** learning

► Given $y^{(i)}$ (true class label of the i th training sample)

Update
values

$$\begin{aligned}\Delta w_j &= \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)} \\ \Delta b &= \eta(y^{(i)} - \hat{y}^{(i)})\end{aligned}$$

$$\begin{aligned}w_j &:= w_j + \Delta w_j \\ b &:= b + \Delta b\end{aligned} \quad j = 1, 2, \dots, m$$

- $(y^{(i)} - \hat{y}^{(i)})$: prediction error
- $\eta \in [0,1]$: **learning rate** (學習率)
- Weight update Δw_j is *proportional to* the value of $x_j^{(i)}$
- Bias unit and all weights are being updated simultaneously

$$z = w_1x_1 + \dots + w_mx_m + b = \mathbf{w}^T\mathbf{x} + b$$

Note that unlike the bias unit, a **feature** $x_j^{(i)}$ is involved in determining the update value Δw_j .

Perceptron Learning Algorithm (PLA)

■ Correct prediction

- ▶ Bias unit and weights remain unchanged

$$y^{(i)} = 0, \quad \hat{y}^{(i)} = 0, \quad \Delta w_j = \eta(0 - 0)x_j^{(i)} = 0, \quad \Delta b = \eta(0 - 0) = 0$$

$$y^{(i)} = 1, \quad \hat{y}^{(i)} = 1, \quad \Delta w_j = \eta(1 - 1)x_j^{(i)} = 0, \quad \Delta b = \eta(1 - 1) = 0$$

■ Wrong prediction

- ▶ Bias unit and weights are being pushed toward the direction of the positive or negative target class

$$y^{(i)} = 1, \quad \hat{y}^{(i)} = 0, \quad \Delta w_j = \eta(1 - 0)x_j^{(i)} = \eta x_j^{(i)}, \quad \Delta b = \eta(1 - 0) = \eta$$

$$y^{(i)} = 0, \quad \hat{y}^{(i)} = 1, \quad \Delta w_j = \eta(0 - 1)x_j^{(i)} = -\eta x_j^{(i)}, \quad \Delta b = \eta(0 - 1) = -\eta$$

Perceptron Learning Algorithm (PLA)

■ Example:

$$z = w_1x_1 + \dots + w_mx_m + b = \mathbf{w}^T \mathbf{x} + b$$

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

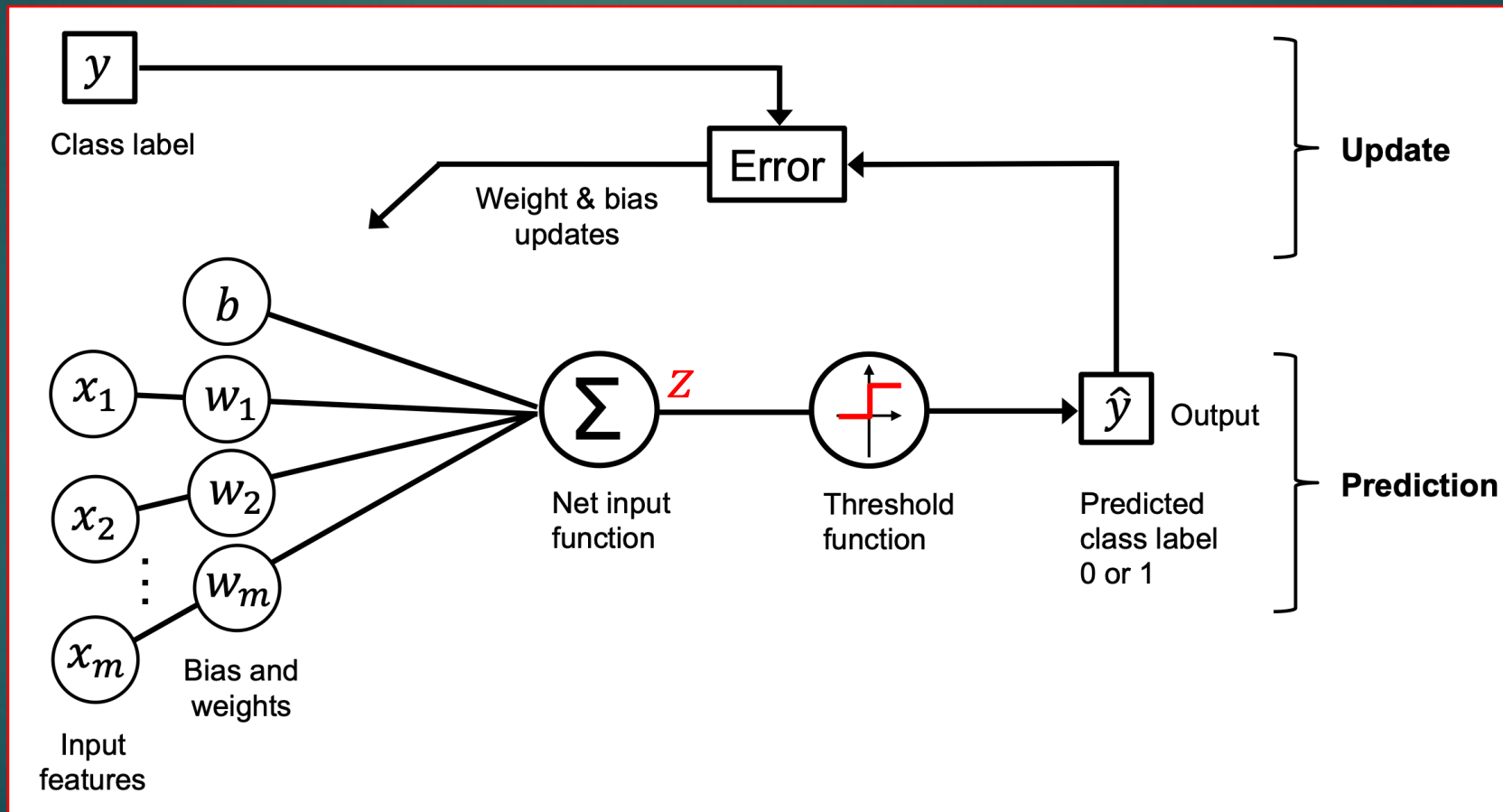
- ▶ Assume that $x_j^{(i)} = 1.5$ and we *misclassify* this example as class 0
- ▶ We would *increase* the corresponding weights and bias unit so that the net input z would be *more positive* the next time we encounter this example (more likely to be classified as class 1)

$$\Delta w_j = (1 - 0)1.5 = 1.5, \quad \Delta b = (1 - 0) = 1$$

$w_j \uparrow \quad b \uparrow$

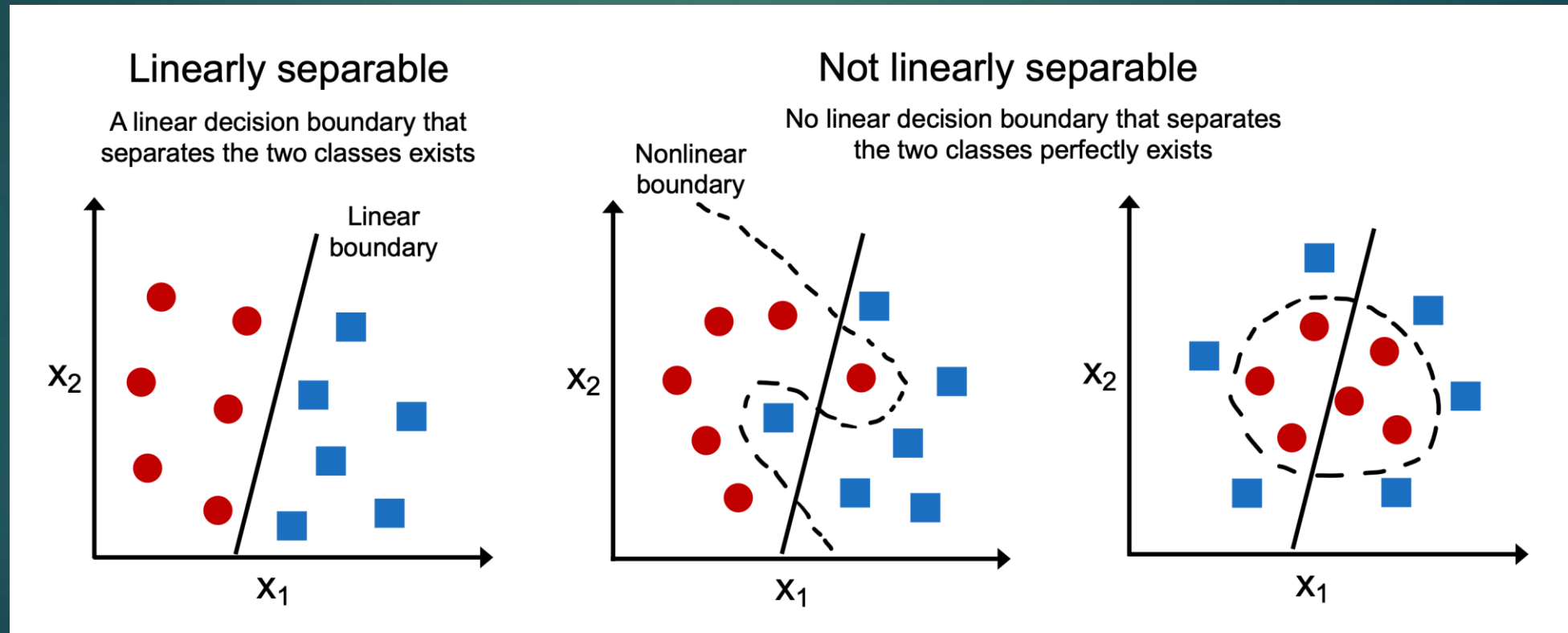
Perceptron Learning Algorithm (PLA)

■ Summary (for each training sample)



Convergence Issue

- **Convergence** is *only* guaranteed if the two classes are **linearly separable**



Convergence Issue

- *If* the two classes are *not* linearly separable...
 - ▶ The perceptron would *never* stop updating the weights
 - ▶ We can set a criterion to “stop training”
 - Maximum number of **epochs** (訓練輪次)
 - A threshold for the number of **tolerated misclassifications**
 - ...

Epoch

- **Epoch**: The *entire dataset* is passed for training *one time* (整個資料集被完整訓練一輪)
- Why do we need multiple epochs? (溫故知新 😲)
 - ▶ A single pass is rarely enough for a model to learn complex patterns (你複習學科只看一次書嗎 😊)
 - ▶ Running multiple epochs allows the model to optimize its weights, but...
 - Too many epochs can lead to **overfitting** (過擬合)
 - Too few can cause **underfitting** (欠擬合)

Extension to Multi-class Classification

- Binary classifier can be extended to **multi-class classification** (多元分類)
- One-versus-all (**OvA**) (also called one-versus-rest (**OvR**))
 - ▶ **Train one classifier per class** (e.g., k classifiers, k is the number of class labels)
 - The particular class is treated as the positive class (label '1')
 - The examples from *all other classes* are considered negative classes
 - ▶ Use k classifiers to predict separately, and assign the class label with the *highest confidence* to the sample
 - In the case of the perceptron, we would use OvA to choose the class label that is associated with the *largest absolute net input value*



Python Basics

Python Basics – 型別轉換與字串格式化

■ 題目：計算圓面積

- ▶ 提示使用者輸入一圓的半徑 (字串)，將其轉換為浮點數
- ▶ 計算圓面積後，輸出一句：「The area of a circle with radius [R] is [Area]」
- ▶ 其中 Area 需四捨五入至小數點後兩位

```
Enter radius: 5
The area of a circle with radius 5.0 is 78.54
```

```
Enter radius: a
Please enter a valid number.
```

■ 複習重點：input() 處理、float() 型別轉換、** 算術運算子、F-string 格式化字串

Python Basics – 型別轉換與字串格式化

■ 參考解答：02_1.py

```
import math

r_str = input("Enter radius: ")
try:
    radius = float(r_str)
    area = math.pi * (radius ** 2)
    print(f"The area of a circle with radius {radius} is {area:.2f}")
except ValueError:
    print("Please enter a valid number.")
```

Python Basics – 資料結構操作

- 題目：給予一清單 `data = [1, 2, 2, 3, 4, 4, 4, 5]`。請分別執行以下操作

- ▶ 移除重複項

- ▶ 計算每個數字出現的次數並存入字典 (Dict)

```
Unique: [1, 2, 3, 4, 5]  
Counts: {1: 1, 2: 2, 3: 1, 4: 3, 5: 1}
```

- 複習重點：set 去重特性、list 轉換、字典的鍵值對操作與 `get()` 方法

Python Basics – 資料結構操作

■ 參考解答：02_2.py

```
data = [1, 2, 2, 3, 4, 4, 4, 5]
# 1. 移除重複
unique_data = list(set(data))

# 2. 計算次數
counts = {}
for item in data:
    counts[item] = counts.get(item, 0) + 1

print(f"Unique: {unique_data}")
print(f"Counts: {counts}")
```

Python Basics – 流程控制與條件邏輯

■ 題目：撰寫一個迴圈

▶ 印出 1 到 20 之間的數字

▶ 但如果是 3 的倍數印出 "Fizz"，5 的倍數印出 "BUZZ"，同時是 3 和 5 的倍數印出 "FizzBUZZ"

```
1 - 2 - Fizz - 4 - Buzz - Fizz - 7 - 8 - Fizz - Buzz - 11 - Fizz - 13 - 14 - FizzBu  
zz - 16 - 17 - Fizz - 19 - Buzz -
```

■ 複習重點：for 迴圈、range() 函式、if-elif-else 多重分支邏輯、取餘數運算子 %

Python Basics – 流程控制與條件邏輯

■ 參考解答：02_3.py

```
for i in range(1, 21):  
    if i % 3 == 0 and i % 5 == 0:  
        print("FizzBuzz", end= " - ")  
    elif i % 3 == 0:  
        print("Fizz", end= " - ")  
    elif i % 5 == 0:  
        print("Buzz", end= " - ")  
    else:  
        print(i, end= " - ")
```


Python Basics – 內建函式與模組應用

■ 題目：使用 `random` 模組

- ▶ 生成一個包含 5 個 1 到 100 之間隨機整數的清單
- ▶ 並使用內建函式找出其中的最大值、最小值與總和

```
List: [80, 33, 31, 51, 5]  
Max: 80, Min: 5, Sum: 200
```

■ 複習重點：import 外部模組、List Comprehension (列表推導式)、`max()` / `min()` / `sum()` 內建函式

Python Basics – 內建函式與模組應用

■ 參考解答：02_4.py

```
import random

nums = [random.randint(1, 100) for _ in range(5)]
print(f"List: {nums}")
print(f"Max: {max(nums)}, Min: {min(nums)}, Sum: {sum(nums)}")
```

Python Basics – 檔案存取、os 模組與例外處理

- 題目：檢查當前目錄下是否存在名為 `test_dir` 的資料夾
 - ▶ 若無則建立它
 - ▶ 若有則使用 `os` 模組列出該資料夾內所有檔案名稱
 - ▶ 請加入 `try-except` 捕捉權限錯誤

```
Created test_dir
```

```
Files in test_dir: ['hello.py', 'test1.py']
```

- 複習重點：`os.path` 檔案路徑判斷、`os.makedirs` 與 `os.listdir`、`try...except` 例外處理

Python Basics – 檔案存取、os 模組與例外處理

■ 參考解答：02_5.py

```
import os

target = "test_dir"
try:
    if not os.path.exists(target):
        os.makedirs(target)
        print(f"Created {target}")
    else:
        files = os.listdir(target)
        print(f"Files in {target}: {files}")
except OSError as e:
    print(f"OS error: {e}")
```

Python Basics – 物件導向程式設計 (OOP)

- 題目：定義一個類別 Point

- ▶ 具有屬性代表座標點 (x, y)

- ▶ 包含一個方法 `distance_from_origin()` 用於計算該點到原點的距離 $\sqrt{x^2 + y^2}$

```
p = Point(3, 4)
print(f"Distance: {p.distance_from_origin()}") # 輸出 5.0
```

- 複習重點：類別定義、`__init__` 建構子 (初始化方法)、`self` 參數的理解、物件實例化與方法呼叫

Python Basics – 物件導向程式設計 (OOP)

■ 參考解答：02_6.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_from_origin(self):
        return (self.x**2 + self.y**2)**0.5

p = Point(3, 4)
print(f"Distance: {p.distance_from_origin()}") # 輸出 5.0
```



Training a Perceptron Model

Mission

- Implement a perceptron (i.e., PLA) in Python
 - ▶ Goal: An **object-oriented perceptron API**
 - `fit(X, y)`: method of learning from data
 - `predict(X)`: method of making predictions
 - ▶ The object-oriented implementation will help us understand the **scikit-learn API**
 - Based on the same core concepts (`fit` and `predict`)
- Apply it to the Iris dataset
- Sample code: **02_7.py**

Import Packages and Modules

■ NumPy

- ▶ 資料科學的基石，Pandas 與 SciPy 等均建構於其上
- ▶ 提供高效能的 N 維陣列 (ndarray) 物件及向量化運算

■ Pandas

- ▶ 資料分析與操作工具庫
- ▶ 提供高效能且易用的資料結構，專門用於處理表格化資料

■ Matplotlib

- ▶ 資料視覺化工具
- ▶ 提供高品質的 2D 與 3D 圖表

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from matplotlib.colors import ListedColormap
```

Object-oriented Perceptron API

■ Define Perceptron class

```
6 # 1.
7 # An object-oriented perceptron API
8 class Perceptron:
9     """Perceptron classifier...
31     def __init__(self, eta=0.01, n_iter=50, random_state=1):
32         self.eta = eta
33         self.n_iter = n_iter
34         self.random_state = random_state
35
36     def fit(self, X, y):
37         """Fit training data...
52         rgen = np.random.RandomState(self.random_state)
53         self.w_ = rgen.normal(loc=0.0, scale=0.01, size=X.shape[1])
54         self.b_ = np.float64(0.)
55
56         self.errors_ = []
57
58         for _ in range(self.n_iter):
59             errors = 0
60             for xi, target in zip(X, y):
61                 update = self.eta * (target - self.predict(xi))
62                 self.w_ += update * xi
63                 self.b_ += update
64                 errors += int(update != 0.0)
65             self.errors_.append(errors)
66         return self
```

$$z = w_1x_1 + \dots + w_mx_m + b = \mathbf{w}^T \mathbf{x} + b$$

```
67
68     def net_input(self, X):
69         """Calculate net input"""
70         #return np.dot(X, self.w_) + self.b_
71         return X @ self.w_ + self.b_
72
73     def predict(self, X):
74         """Return class label after unit step"""
75         return (self.net_input(X) >= 0.0).astype(int)
```

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\Delta w_j = \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$$
$$\Delta b = \eta(y^{(i)} - \hat{y}^{(i)})$$

$$w_j := w_j + \Delta w_j$$
$$b := b + \Delta b$$

Initialization of the Weights

- Initial weight vector contains *small random numbers* $\sim N(0, 0.01)$
- Can we initialize the weights to zero?
 - ▶ Yes, but NOT recommended!
 - Learning rate η would have no effect on the decision boundary (affects only the scale of the weight vector, not the direction)

$$\text{學習率 A: } w_1^A = 0 + \eta_A \cdot \text{error}_1 \cdot x_1 = \boldsymbol{\eta}_A \cdot (\text{error}_1 \cdot x_1)$$

$$\text{學習率 B: } w_1^B = 0 + \eta_B \cdot \text{error}_1 \cdot x_1 = \boldsymbol{\eta}_B \cdot (\text{error}_1 \cdot x_1)$$

$$\text{學習率 A: } w_2^A = w_1^A + \eta_A \cdot \text{error}_2 \cdot x_2 = \boldsymbol{\eta}_A \cdot (\text{error}_1 \cdot x_1) + \boldsymbol{\eta}_A \cdot (\text{error}_2 \cdot x_2) = \boldsymbol{\eta}_A \cdot [(\text{error}_1 \cdot \mathbf{x}_1) + (\text{error}_2 \cdot \mathbf{x}_2)]$$

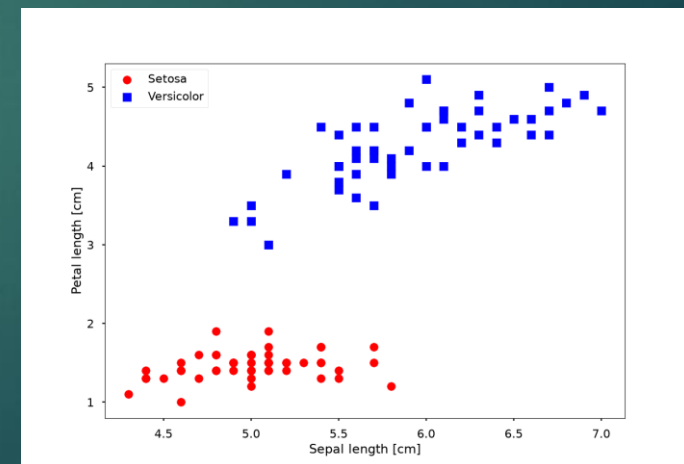
$$\text{學習率 B: } w_2^B = w_1^B + \eta_B \cdot \text{error}_2 \cdot x_2 = \boldsymbol{\eta}_B \cdot (\text{error}_1 \cdot x_1) + \boldsymbol{\eta}_B \cdot (\text{error}_2 \cdot x_2) = \boldsymbol{\eta}_B \cdot [(\text{error}_1 \cdot \mathbf{x}_1) + (\text{error}_2 \cdot \mathbf{x}_2)]$$

Load the Iris Dataset

■ Load and plot the data

```
80 # 2.
81 # Reading-in the Iris data
82 s = 'iris.data'
83 df = pd.read_csv(s, header=None, encoding='utf-8')
84
85 # Target array: select 2 classes only
86 # setosa (class '0') and versicolor (class '1')
87 y = df.iloc[0:100, 4].to_numpy()
88 y = np.where(y == 'Iris-setosa', 0, 1)
89
90 # Feature matrix: select 2 features only
91 # sepal length and petal length
92 X = df.iloc[0:100, [0, 2]].to_numpy()
```

```
94 # plot data
95 plt.scatter(X[:50, 0], X[:50, 1],
96            color='red', marker='o', label='Setosa')
97 plt.scatter(X[50:100, 0], X[50:100, 1],
98            color='blue', marker='s', label='Versicolor')
99 plt.xlabel('Sepal length [cm]')
100 plt.ylabel('Petal length [cm]')
101 plt.legend(loc='upper left')
102 plt.show()
```



A linear decision boundary *should* be sufficient to separate setosa from versicolor flowers

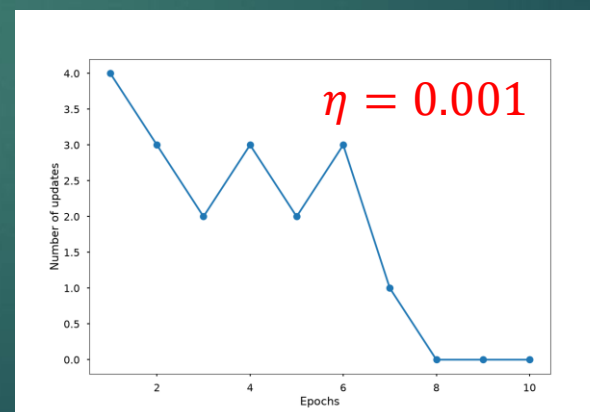
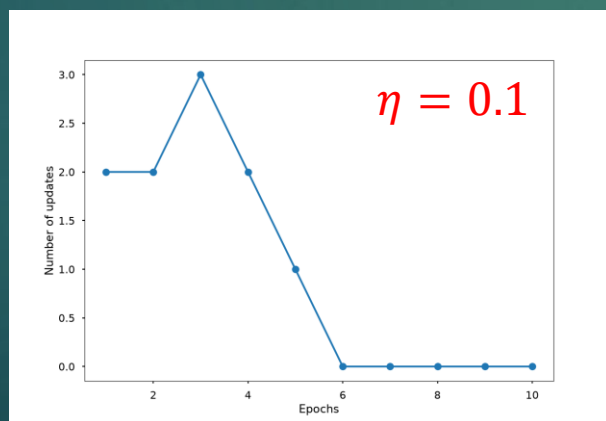
Load the Iris Dataset

- Perceptron is a *binary* classifier
 - ▶ Only consider *two* flower classes from the Iris dataset
 - Setosa
 - Versicolor
- To *visualize* the decision regions of the trained model
 - ▶ Only select *two* feature variables (dimensions)
 - Sepal length
 - Petal length
 - ▶ Although the perceptron rule is *not* restricted to two dimensions

Training the Perceptron Model

- Train the model & plot the training curve (訓練曲線)

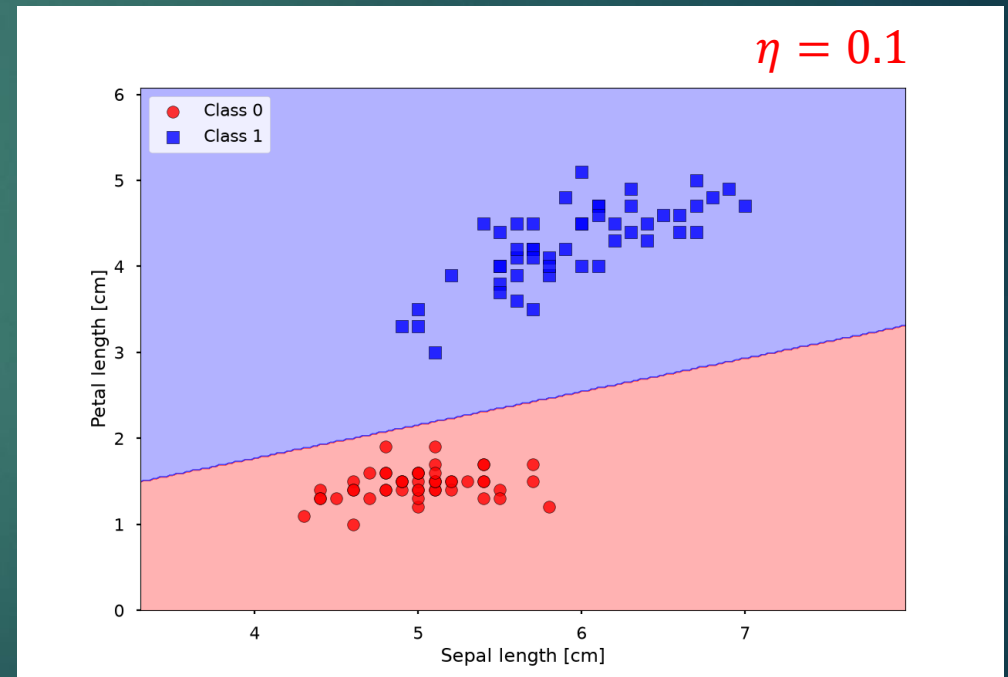
```
105 # 3.  
106 # Training the perceptron model on the Iris dataset  
107 ppn = Perceptron(eta=0.1, n_iter=10)  
108 ppn.fit(X, y)  
109  
110 plt.plot(range(1, len(ppn.errors_) + 1), ppn.errors_, marker='o')  
111 plt.xlabel('Epochs')  
112 plt.ylabel('Number of updates')  
113 plt.show()
```



Visualization of the Decision Boundary

```
116 # 4.
117 # Visualization of the decision boundary
118 # A utility function for plotting decision regions
119 def plot_decision_regions(X, y, classifier, resolution=0.02):
120     # setup marker generator and color map
121     markers = ('o', 's', '^', 'v', '<')
122     colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
123     cmap = ListedColormap(colors[:len(np.unique(y))])
124
125     # plot the decision surface
126     x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
127     x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
128     xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
129                             np.arange(x2_min, x2_max, resolution))
130     lab = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
131     lab = lab.reshape(xx1.shape)
132     plt.contourf(xx1, xx2, lab, alpha=0.3, cmap=cmap)
133     plt.xlim(xx1.min(), xx1.max())
134     plt.ylim(xx2.min(), xx2.max())
135
136     # plot class examples
137     for idx, cl in enumerate(np.unique(y)):
138         plt.scatter(x=X[y == cl, 0],
139                     y=X[y == cl, 1],
140                     alpha=0.8,
141                     c=colors[idx],
142                     marker=markers[idx],
143                     label=f'Class {cl}',
144                     edgecolor='black')
```

```
146 plot_decision_regions(X, y, classifier=ppn)
147 plt.xlabel('Sepal length [cm]')
148 plt.ylabel('Petal length [cm]')
149 plt.legend(loc='upper left')
150 plt.show()
```





Thank you